# Dynamic Tables


Powered by

www.learn2clouddata.com

Sujith Nair

Cloud Data Architect
Snowflake Snowpro Certified

# Section Introduction-Dynamic Tables Topics.

- What are Dynamic tables ?
- Dynamic table refresh.
- LAB 1-Create Dynamic tables.
- Costs of Dynamic tables
- Limitation of Dynamic tables.
- Dynamic table pipeline.
- LAB 2-Create Dynamic table pipeline.
- How to monitor Dynamic tables.
- LAB 3-Monitor Dynamic tables.
- Summary.

# What are Dynamic tables?

Dynamic tables are similar to views and has a query associated with it which it uses to select data from one or more tables called base tables and refresh the Dynamic table based on the query. Dynamic tables store data physically in them unlike views.

Dynamic tables features.

- Physically store data in the table
- Query is used to populate the Dynamic table and keep it in sync with the base tables .
- Dynamic tables are refreshed based on a parameter called TARGET_LAG.
- TARGET_LAG can be in seconds, minutes, hours or days.
- TARGET_LAG={seconds | minutes | hours| days} eg: TARGET_LAG='10 minutes'.
- TARGET_LAG='DOWNSTREAM' manually refresh Dynamic table with ALTER DYNAMIC table <tablename> REFRESH.
- Dynamic table pipeline is created when dynamic tables are chained together with one Dynamic table being dependent on the other, we can have at the max 10 Dynamic tables in a pipeline.

# What are Dynamic tables?

Drawbacks of Views:

- Views can be slow for very large Tables
- View Query will consume lot of credits

Dynamic tables:

- Select performance of Dynamic table is better than views.
- Dynamic table Query is used to populate the Dynamic table and keep it in sync.
- Multiple Tables can be used in a Dynamic table.

# Dynamic table Create Syntax

```
CREATE OR REPLACE DYNAMIC TABLE DYT_EMP_DEPT
TARGET_LAG='10 minutes'
WAREHOUSE ='COMPUTE_WH'
AS
SELECT
        E.EMPLOYEE_ID,
        E.JOB_ID,
        E.MANAGER_ID,
        E.DEPARTMENT_ID
FROM      EMPLOYEES E JOIN DEPARTMENTS D
ON E.DEPARTMENT_ID=D.DEPATMENT_ID
```

# Dynamic table Refresh

Dynamic tables refresh only if there is a change in the underlying data, this helps reduce unnecessary credit consumption which happens in the case of Tasks which run at a fixed interval irrespective of if the base tables have undergone any data change.


Dynamic tables support 2 refresh types

- Incremental Refresh
- Full Refresh

REFRESH_MODE = { AUTO | FULL | INCREMENTAL }

When REFRESH_MODE is set to AUTO Snowflake will attempt an incremental refresh .Incremental refresh consumes less credits as compared to full refresh. If incremental refresh is not possible then Snowflake will attempt full refresh.

When REFRESH_MODE is set to FULL Snowflake will perform a full refresh irrespective of if it is possible to do an incremental refresh.

When REFRESH_MODE is set to INCREMENTAL Snowflake will attempt  an incremental refresh and if it is not able to perform an incremental refresh then the refresh will fail.

Some SQL constructs when included in the Dynamic table query do not allow Snowflake to perform incremental refresh. SET operators , PIVOT, UNPIVOT LATERAL JOIN,IN-EQUALITY OPERATOR in OUTER JOIN.

# Dynamic table Refresh


We can manually refresh Dynamic tables by running
ALTER DYNAMIC TABLE <Tablename> REFRESH;
This can be run irrespective of what the TARGET_LAG parameter is set to.

We can suspend Dynamic tables by using
ALTER DYNAMIC TABLE <Tablename> SUSPEND;
We may need to suspend Dynamic tables if we are going to conduct maintenance operations on the tables in the Dynamic tables. If we suspend a Dynamic table, all Dynamic tables that are upstream from this Dynamic table will be suspended which means suspension cascades.


We can resume a suspended Dynamic table by using
ALTER DYNAMIC TABLE <Tablename> RESUME;

Business requirement changes and you may need to change value of TARGET_LAG and this can be done with the
ALTER DYNAMIC TABLE SET
TARGET_LAG ='10 minutes'
WAREHOUSE=COMPUTE_WH;

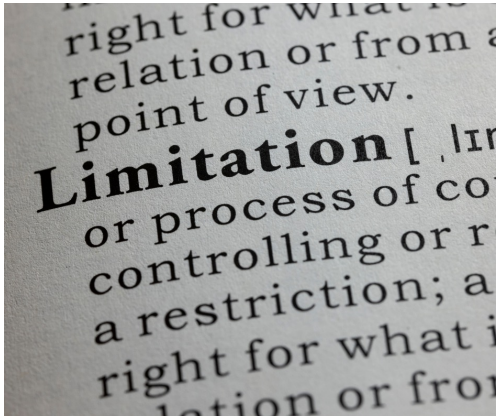
A Dynamic table is auto-suspended if the system observes five continuous refresh errors.

# Costs of Dynamic tables.

Dynamic tables incur cost in the following 3 ways

- Storage cost.
- Virtual warehouse cost.
- Cloud services compute cost.

# Dynamic table limitations.




- A single account can hold a maximum of 1000 Dynamic tables.
- Cannot truncate, update or delete a Dynamic table.
- Cannot create a temporary Dynamic table.
- You cannot use secondary roles with Dynamic tables.
- Query acceleration service is not supported on Dynamic tables.
- Cannot enable search optimization on Dynamic tables.
- Dynamic table SQL Limitations
    - Cannot use External functions.
    - Cannot use CURRENT_USER or functions that use CURRENT_USER.
    - Cannot use Views created on other Dynamic tables.
    - Cannot use UDF or UDTF written in SQL and containing a subquery.
    - Cannot use sources that include  Streams and Materialized views.
    - Cannot use sources that include External table, Directory table, Iceberg tables.

# Dynamic table limitations.

- In Dynamic table definition
  - Cannot have more than 100 tables.
  - Cannot query more than 10 Dynamic tables.
  - Cannot chain more than 10 Dynamic tables to create a DAG

# How to monitor Dynamic tables.

To monitor Dynamic tables and locate refresh problems we can use INFORMATION_SCHEMA table functions.

DYNAMIC_TABLE_REFRESH_HISTORY provides the history of refreshes for one or more Dynamic tables in the account. To identify the refreshes that had errors pass the parameter ERROR_ONLY=>TRUE

```
SELECT * FROM TABLE (INFORMATION_SCHEMA.DYNAMIC_TABLE_REFRESH_HISTORY
(
NAME_PREFIX=>'HRMS.HR',ERROR_ONLY=>FALSE,
DATA_TIMESTAMP_START =>TO_TIMESTAMP_LTZ('2024-03-07 07:00:38','YYYY-MM-DD HH24:MI:SS')
)
                    );
```

```
SELECT * FROM TABLE (INFORMATION_SCHEMA.DYNAMIC_TABLE_REFRESH_HISTORY
(
NAME=>'HRMS.HR.DYT_EMP_DEPT_LOC_COUNTRY_REGION_JOB_HIST_JOB',ERROR_ONLY=>FALSE,
DATA_TIMESTAMP_START =>TO_TIMESTAMP_LTZ('2024-03-07 07:00:38','YYYY-MM-DD HH24:MI:SS')
)
);
```

DYNAMIC_TABLE_GRAPH_HISTORY

returns information on all Dynamic tables in the current account.
This information includes dependencies between Dynamic table and on base tables.
A common use is to identify all Dynamic tables that are part of a pipeline. The output also reflects changes made to the properties of a dynamic table over time. Each row represents a dynamic table and a specific set of properties.

If you change a property of a dynamic table (for example, the target lag), the function produces a new row of output for that updated set of properties.

```
SELECT
    *
FROM
    TABLE (INFORMATION_SCHEMA.DYNAMIC_TABLE_GRAPH_HISTORY
    (
    HISTORY_START =>TO_TIMESTAMP_LTZ('2024-03-07 07:00:38', 'YYYY-MM-DD HH24:MI:SS')
    )

          );
```

# Dynamic table pipelines.

Dynamic tables can be used to create data transformation pipelines by defining one dynamic table over the other. Data from one Dynamic table becomes the source for the next Dynamic table creating a pipeline of Dynamic tables which are dependent on each other.

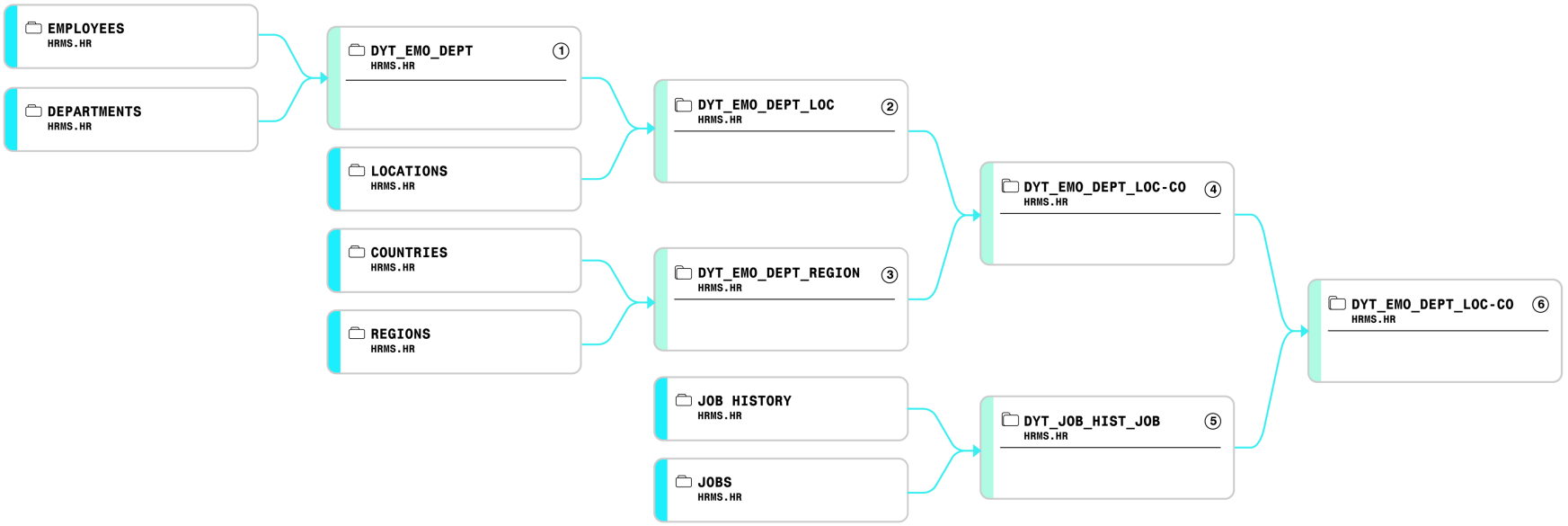
Data can be transformed in the SELECT query of the Dynamic table and this transformed data can then be consumed by the next Dynamic table in the pipeline.

Dynamic tables refresh if there is change in the underlying data and would skip the refresh if there is no change.


When creating a data pipeline we should set the TARGET_LAG=Downstream for all Dynamic tables except the final Dynamic table on which no other Dynamic tables are dependent.
The final Dynamic table should be given a TARGET_LAG based on the refresh rate desired.
Snowflake will refresh all the other Dynamic tables in the pipeline based on the TARGET_LAG of the final table.

# Dynamic Table Pipelines.

# DYNAMIC TABLE PIPELINE



# Summary

- Dynamic tables are like views and have a query associated with them.
- Dynamic tables physically store data and data is refreshed with the query associated with it.
- Dynamic table refresh is controlled by the parameter TARGET_LAG.
- Dynamic tables do not refresh if the underlying tables do not have any data changes.
- Dynamic tables refresh incrementally or full refresh depending on the SQL used and REFRESH_MODE. REFRESH_MODE = { AUTO | FULL | INCREMENTAL }
- Dynamic tables can be forced to refresh by running ALTER DYNAMIC TABLE <tablename> REFRESH.
- Dynamic tables can be suspended by running ALTER DYNAMIC TABLE <tablename> SUSPEND.
- Dynamic tables can be resumed by running ALTER DYNAMIC TABLE <tablename> RESUME.
- Dynamic tables TARGET_LAG can be changed by using ALTER DYNAMIC TABLE SET TARGET_LAG=<NewValue>.
- A Dynamic table is auto-suspended if the system observes five continuous refresh errors.
- Dynamic tables consume storage, compute and cloud services cost.
- Dynamic tables have a lot of limitations and we need to be aware of them.
- There can be a maximum of 10 Dynamic tables in a pipeline.
- Dynamic tables can be used to create data pipelines to transform data and keep the data refreshed at regular intervals.
- We can monitor Dynamic tables with DYNAMIC_TABLE_REFRESH_HISTORY and DYNAMIC_TABLE_GRAPH_HISTORY.
- TARGET_LAG and WAREHOUSE are mandatory parameters when creating a Dynamic table.

Thank you


Powered by

www.learn2clouddata.com

## Find Us

www.learn2clouddata.com

+01 234 567 89